MS2 Summary for MLOPS Project

Theodora Haimoff

Project Title	Swiss Segmented Unemployment Rate Predictor (P04)
Milestone	MS2 · Feature Pipeline
Repository	https://github.com/theodorahaimoff/HSLU_HS26_MLOPS
Decision log	`docs/decisions.md` (D1–D14), referenced below as Dn
Date of Submission	02.11.2026

1 What I Achieved in this Milestone

The feature pipeline runs on a schedule without manual steps. Every day it pulls three live sources, stores every download with its date and hash, rebuilds a point-in-time correct feature table for all 45 series (national total and 44 gender × nationality × age segments) and writes it as a new version to a versioned Parquet feature store on GCS.

Live ingestion & backfill	SAKE (BFS, quarterly target) via the `opendata.swiss` CKAN API, SECO Weekly Economic Activity (weekly) and LIK inflation from the SNB data portal (monthly). Each run recomputes the full backfill: one row set per SAKE release since 2011–Q1 (62 release dates × 45 series, the MS3 training data) plus *refresh* rows as of the run date, the current nowcast.
Features without leakage	Every value gets an `available_from` date (release date, or period end + publication lag). A row may only use values available on its `as_of` date. Three checks run before anything is written; the strongest rebuilds random dates after deleting all later data and requires identical features.
Feature store	GCS bucket: `raw/pull_log.csv` (download date, SHA-256, release date), raw files stored only when their content changed, and `feature_store/features/version=<date>/` with `features.parquet` and a `manifest.json` (source hashes, git commit, lags).
Scheduled execution	GitHub Actions `feature_pipeline.yml`, daily at 05:17 UTC (weekly until 02.11, D12). Scheduled runs on 26.10 and 02.11, daily from 03.11, all green; tests run first, so a broken pipeline writes no version.
Docker, pinned deps, README	`Dockerfile`: `python:3.14-slim`, uv pinned, dependency layer first, non-root user, credentials only at run time. All installs use `uv sync --locked` on `uv.lock`. README: architecture, data, feature store layout, how to run locally, in Docker and in CI.
Tests, CI & secrets	30 unit tests on small fixture files in the exact source formats, so they run in CI without network or credentials; `ci.yml` runs lint, tests and the Docker build on every push. On the real data, source validation (structure and value ranges, D14) and the leakage checks run in every scheduled run before a version is written. Keys live in `.env` (gitignored) and GitHub Secrets; the history contains none.

1.1 Key decisions

- **Nowcast instead of a static quarterly pipeline** (D1–D3): SAKE stays the target, weekly WEA and monthly inflation make the features change between releases.
- **Target = year-on-year change** (D6): SAKE is not seasonally adjusted (Q1 and Q3 lie about 0.3 pp above the yearly mean, Q2 and Q4 below).
- **Point-in-time features at the SAKE release date** (D7), with publication lags observed in the data (SAKE 49 days, WEA $\leq$ 24, LIK 21) and chosen on the late side: too late is pessimistic, too early would leak.
- **Versioning in the pipeline instead of DVC** (D11): data arrives on a schedule, so every pull is hashed and stored at download time, and every feature version names its source hashes and git commit.

Observed on the way: SECO revised past WEA values between my pulls of 15.10 and 25.10 (2026–Q1 mean $-0.41 \rightarrow -0.49$), which is exactly why every pull is kept with its date.

1.2 What a reviewer should look at first

1. `notebooks/02_feature_pipeline.ipynb`: every step of the pipeline with its outputs and findings.

2. `src/feature_pipeline/features.py` builds the features (`build_features`, `backfill`) and checks them for leakage (`check_features`); `tests/test_features.py` proves it on every date.
3. The **Actions** tab: scheduled runs of *Feature pipeline*, and *ci* on every push.
4. Run it yourself, no credentials needed; this pulls the live sources and writes the feature store to `data/`: `uv sync --locked && uv run python -m src.feature_pipeline.run`

2 Response to Reviews

Every labelled point from both MS1 reviews and the editorial summary; details and alternatives are in the decision log.

Label	Point	Status	Answer · where
E1 (Must)	Data too static: SAKE changes once a quarter, so ingest → retrain → monitor can hardly be shown	**Changed**	Nowcast design: SAKE stays the quarterly target; SECO WEA (weekly) and monthly LIK inflation were added as live features. New WEA or LIK values reach the feature store as refresh rows on the next daily run, so the forecast input changes every week. A monthly SECO target was rejected: no API, monthly figures only as PDF (would need scraping). D1–D3, D12 · `src/feature_pipeline/sources.py`
E2	Weekly scheduled retraining on unchanged quarterly labels produces identical models	**Partly**	Decided: retrain on a new SAKE release, not on a schedule (D4). The trigger signal exists now: a new SAKE hash in the pull log sets `new_sake_release` in the manifest of that feature version. The retraining workflow that reacts to it is MS3 work. `src/feature_pipeline/pull_log.py`
A1	Release detection by the hash of one fixed file URL misses releases published as new resources; store release dates	**Changed**	Each run lists the dataset's resources through CKAN and takes the CSV with the latest `issued` date, not a fixed URL. The pull log stores download date, release date, latest quarter and SHA-256 per pull. Tested: a newer release as a separate resource is picked (`tests/test_sources.py`), and two consecutive pulls store only what changed (`tests/test_storage.py`). The first real new release (2026–Q3, expected around 18.11) is the live check. D5 · `src/feature_pipeline/sources.py` (`pull_sake`)
A2, B1	Small segments: how does BFS mark unreliable or suppressed values, and how is it shown?	**Partly**	BFS marks reliability in a separate `STATUS` column (A normal, U low reliability, O missing), not with brackets. In scope there are no O values, but 17 of 45 segments have U quarters (7 in more than half). The code is kept as feature `status_t`; the success target is measured on the 28 segments without U and the others are reported separately. The UI flag follows with the UI in MS4. D9 · `notebooks/01_data_exploration.ipynb`, sections 2–3
B2	15 % target against seasonal naive only; persistence may be the stronger baseline	**Changed**	Both computed on the stored features, last 12 target quarters, MAE in pp: seasonal naive 0.794 vs. persistence 0.976 over all 45 series, and 0.630 vs. 0.672 on the 28 reliable segments. Seasonal naive is stronger in every group (persistence carries last quarter's season into the next). Restated target: MAE $\leq$ 0.535 pp on the 28 reliable segments, 15 % below seasonal naive. D10 · `src/training_pipeline/baselines.py`
B3	`requirements.txt` with minimum versions only	**Changed**	`uv.lock` pins every package with version and hash; CI, the scheduled job and the Docker build install with `uv sync --locked`, which fails on a stale lockfile. `requirements.txt` removed. Dev tools, training and UI packages are separate groups, so the image holds only the 5 packages the feature pipeline needs. D8 · `pyproject.toml`, `uv.lock`, `Dockerfile`
E3	Why FastAPI *and* Streamlit?	**Changed**	FastAPI dropped. The forecasts are batch-shaped: all inputs are in the feature store before anyone asks, and all 45 series are forecast per run, so an API would only return what the batch job already computed. A single Streamlit app on Cloud Run will read the forecasts from GCS. Model version, feature time and creation time are stored per forecast row and shown in the UI. D13 · `docs/decisions.md`